

J Unit Testing Exercises

Exercise 1: Setting Up JUnit

Scenario:

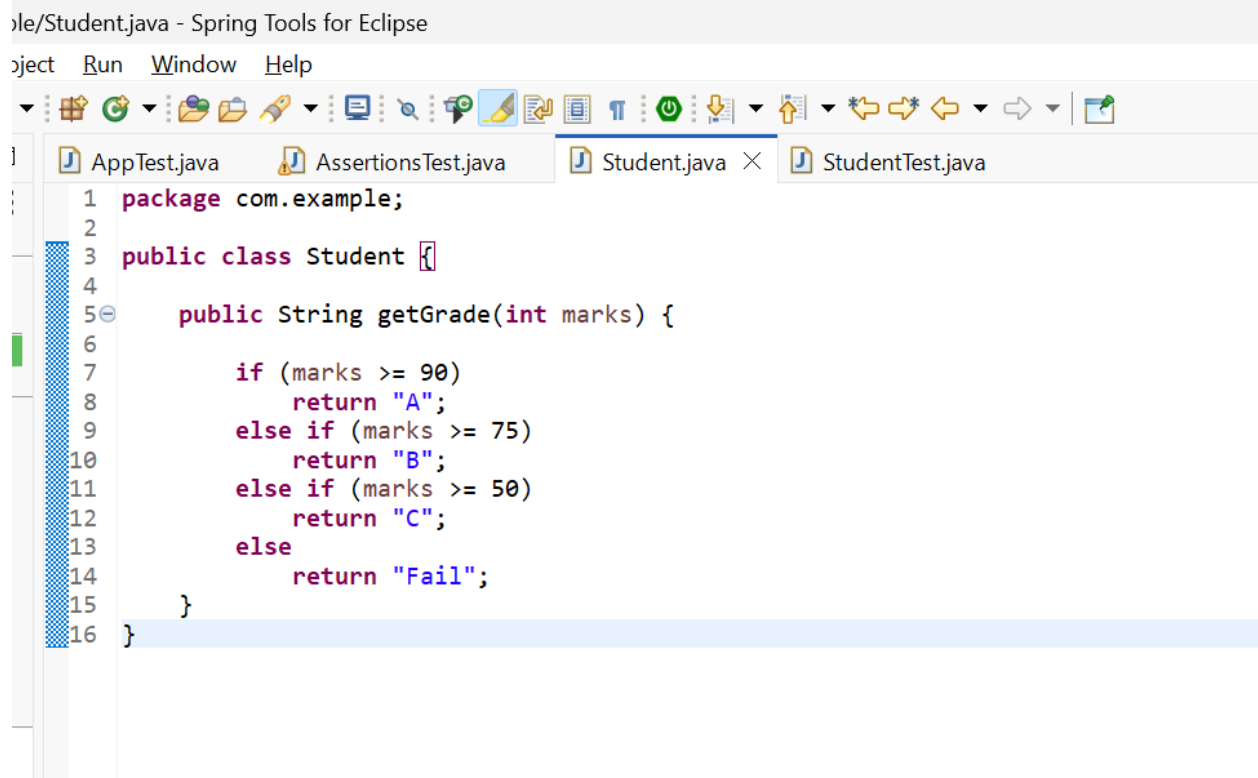
You need to set up JUnit in your Java project to start writing unit tests.

Steps:

1. Create a new Java project in your IDE (e.g., IntelliJ IDEA, Eclipse).
2. Add JUnit dependency to your project. If you are using Maven, add the following to your pom.xml:

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13.2</version>
  <scope>test</scope>
</dependency>
```

Student.java

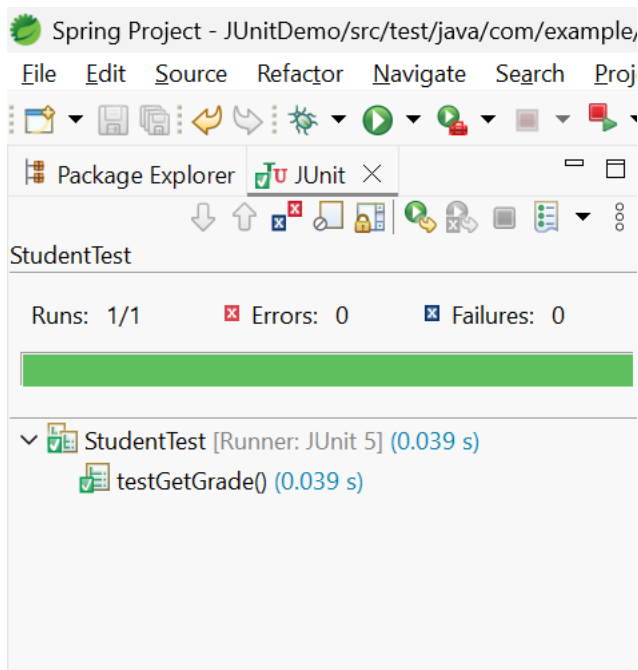


```
le/Student.java - Spring Tools for Eclipse
ject  Run  Window  Help
AppTest.java  AssertionsTest.java  Student.java  StudentTest.java
1  package com.example;
2
3  public class Student {
4
5      public String getGrade(int marks) {
6
7          if (marks >= 90)
8              return "A";
9          else if (marks >= 75)
10             return "B";
11          else if (marks >= 50)
12             return "C";
13          else
14             return "Fail";
15      }
16  }
```

StudentTest.java

```
AppTest.java  AssertionsTest.java  Student.java  StudentTest.java ×
1 package com.example;
2
3 import static org.junit.jupiter.api.Assertions.assertEquals;
4 import org.junit.jupiter.api.Test;
5
6 public class StudentTest {
7
8     @Test
9     public void testGetGrade() {
10
11         Student student = new Student();
12
13         assertEquals("A", student.getGrade(95));
14         assertEquals("B", student.getGrade(80));
15         assertEquals("C", student.getGrade(60));
16         assertEquals("Fail", student.getGrade(40));
17     }
18 }
```

Getting output as:



Exercise 3: Assertions in JUnit

Scenario:

You need to use different assertions in JUnit to validate your test results.

Write tests using various JUnit assertions.

Answer:

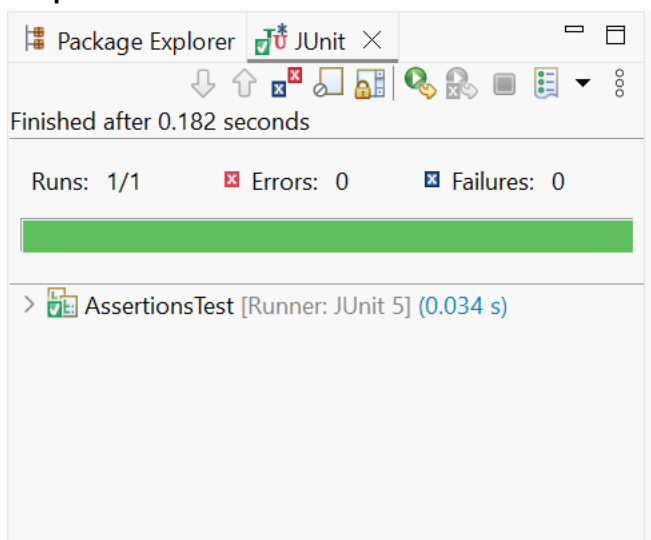
AssertionDemo.java

```
AppTest.java Student.java AssertionDemo.java X AssertionsTest.java
1 package com.example;
2
3 public class AssertionDemo {
4
5     public int add(int a, int b) {
6         return a + b;
7     }
8
9     public String getMessage() {
10        return "Welcome";
11    }
12
13    public Object getObject() {
14        return new Object();
15    }
16
17    public Object getNullObject() {
18        return null;
19    }
20
21    public boolean isEligible(int age) {
22        return age >= 18;
23    }
24 }
```

AssertionsTest.java

```
AppTest.java Student.java AssertionDemo.java AssertionsTest.java X
1 package com.example;
2
3 import static org.junit.jupiter.api.Assertions.*;
4
5 import org.junit.jupiter.api.Test;
6
7 public class AssertionsTest {
8
9     AssertionDemo demo = new AssertionDemo();
10
11     @Test
12     public void testAssertions() {
13
14         assertEquals(10, demo.add(4, 6));
15
16
17         assertNotEquals(15, demo.add(4, 6));
18         assertTrue(demo.isEligible(20));
19         assertFalse(demo.isEligible(15));
20         assertNull(demo.getNullObject());
21         assertNotNull(demo.getObject());
22         String s1 = "Java";
23         String s2 = s1;
24         assertSame(s1, s2);
25         String s3 = new String("Java");
26         String s4 = new String("Java");
27         assertNotSame(s3, s4);
28     }
29 }
```

Output:



Exercise 4: Arrange-Act-Assert (AAA) Pattern, Test Fixtures, Setup and Teardown Methods in JUnit

Scenario:

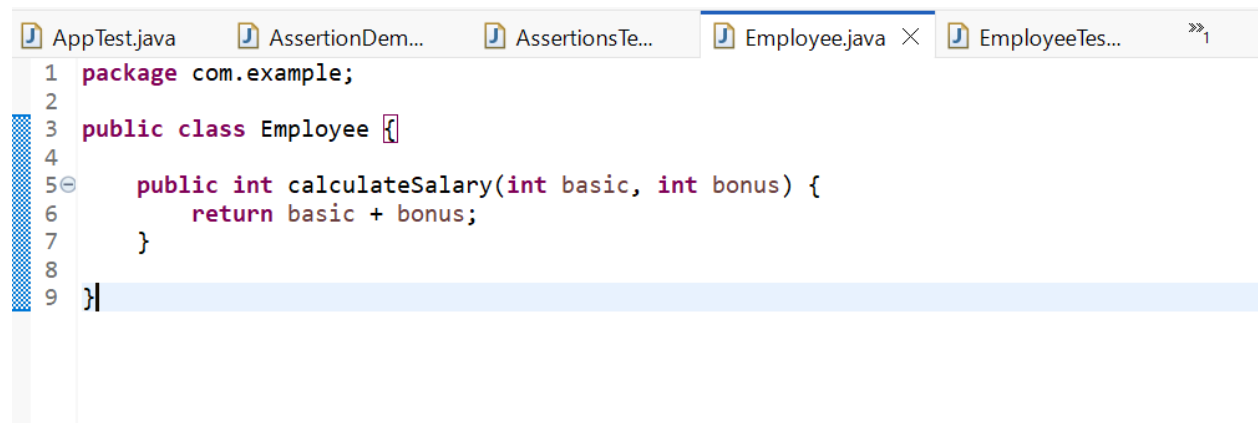
You need to organize your tests using the Arrange-Act-Assert (AAA) pattern and use setup and teardown methods.

Steps:

1. Write tests using the AAA pattern.
2. Use `@Before` and `@After` annotations for setup and teardown methods.

Solution:

Employee.java



```
1 package com.example;
2
3 public class Employee {
4
5     public int calculateSalary(int basic, int bonus) {
6         return basic + bonus;
7     }
8
9 }
```

EmployeeTest.java

```
AppTest.java  AssertionDem...  AssertionsTe...  Employee.java  EmployeeTes...  »1
1 package com.example;
2
3 import static org.junit.jupiter.api.Assertions.assertEquals;
4
5 import org.junit.jupiter.api.AfterEach;
6 import org.junit.jupiter.api.BeforeEach;
7 import org.junit.jupiter.api.Test;
8
9 public class EmployeeTest {
10     Employee employee;
11     @BeforeEach
12     public void setUp() {
13         System.out.println("Setup Method");
14         employee = new Employee();
15     }
16     @AfterEach
17     public void tearDown() {
18         System.out.println("Teardown Method");
19         employee = null;
20     }
21     @Test
22     public void testCalculateSalary() {
23         int basic = 25000;
24         int bonus = 5000;
25         int salary = employee.calculateSalary(basic, bonus);
26         assertEquals(30000, salary);
27     }
28 }
```

Output:

```
Package Explorer  JUnit  X
Finished after 0.119 seconds
Runs: 1/1  Errors: 0  Failures: 0
> EmployeeTest [Runner: JUnit 5] (0.022 s)
```